Beginner's Guide: Your First Step into Git & GitHub

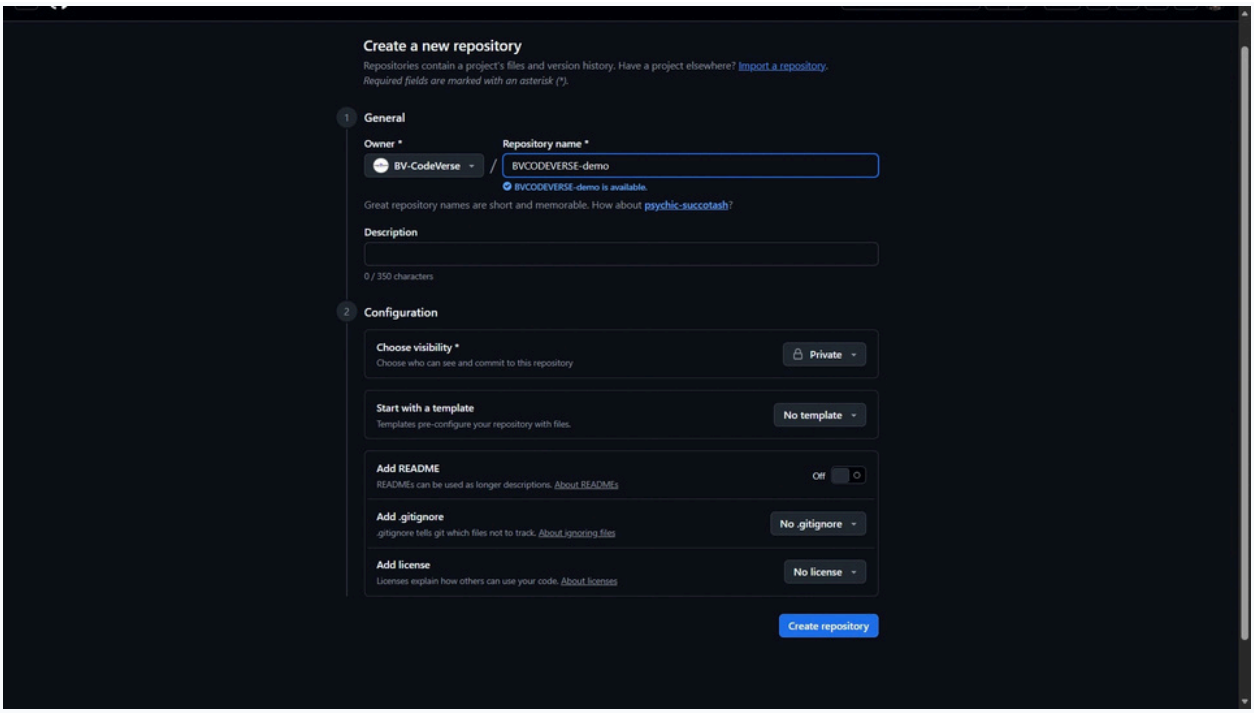
Part 1: What is Git & GitHub? (And the GUI)

Imagine writing a massive school project or coding assignment. Every time you make changes, you probably save files like **project_v1.zip**, **project_v2.zip**, or **final_final_real.zip**. It gets messy quickly!

- **What is Git?** Git is like a **time machine** for your code. It runs on your computer and keeps track of every single change you make. If you break your code, you can instantly travel back in time to when it was working.
- **What is GitHub?** If Git is the time machine on your laptop, **GitHub** is the cloud storage (like Google Drive or iCloud) where you upload your project so it is safe online and you can share it with others.
- **What is the GitHub GUI? GUI** stands for *Graphical User Interface*. Instead of typing scary computer commands, the GUI is GitHub's website where you can click buttons, view files, and create projects using your mouse.

Key GitHub GUI Actions:

1. **How to create a Repository (Repo):** A "repository" is just a fancy word for a project folder. To create one, go to GitHub, click the **+** icon in the top right corner, select **New repository**, give it a name, check the box that says **"Add a README file"**, and click **Create repository**.
2. **What is a Branch?** Think of a branch as a copy of your project. The main version is called main. If you want to try adding a cool new feature without breaking your working project, you create a new "branch" to experiment safely.



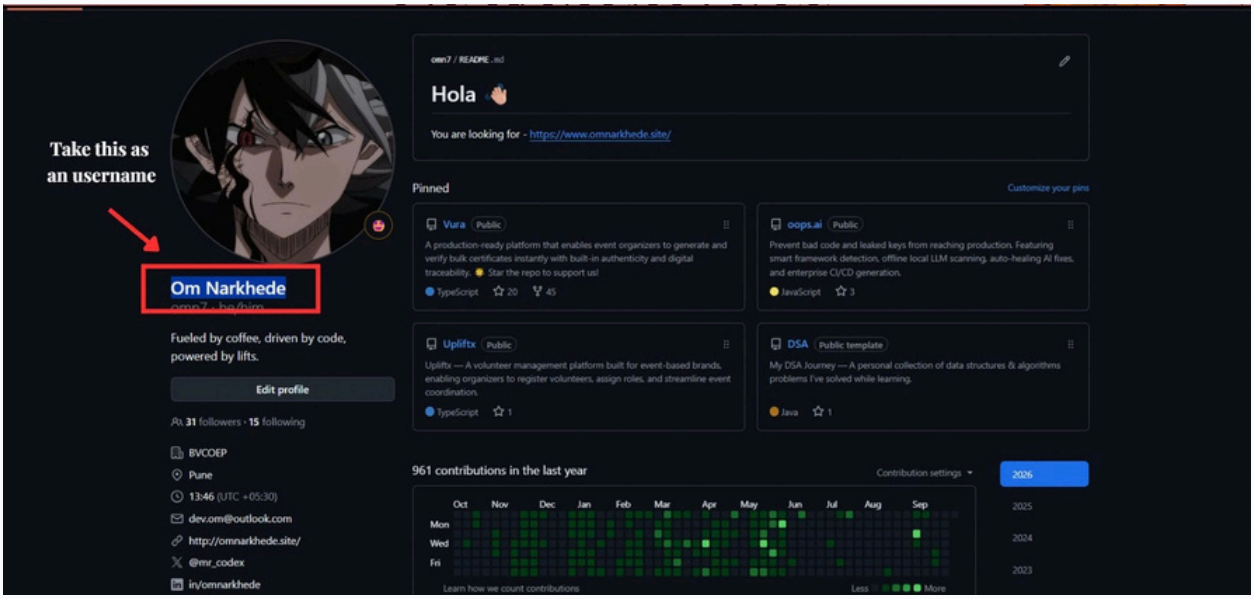
Part 2: Introduction to Git & Setting Up Your Computer

To use Git on your computer, you will use a tool called **Git Bash** (a window where you type short commands).

Step 1: Tell Git Who You Are (Do this once)

Open Git Bash on your computer and type these two lines (with your own name and email) to let Git know who is writing the code:

```
git config --global user.name "Your Full Name"
git config --global user.email "your.email@example.com"
```



Step 2: The Core Commands Cheat Sheet

When working with Git, you will use these commands over and over again:

- **git init**: Turns your ordinary computer folder into a Git-tracked project.
- **git status**: Asks Git, *"What files have I changed or added recently?"*
- **git add .**: Selects and "stages" your files, telling Git: *"Get these files ready to be saved."*
- **git commit -m "your message"**: Saves your staged files permanently into Git's history. Think of the message as a sticky note explaining what you changed (e.g., "added login page").
- **git remote add origin <link>**: Connects your computer folder to your online GitHub repository.
- **git push**: Uploads your saved changes from your computer up to GitHub.
- **git pull**: Downloads the newest updates from GitHub down to your computer.

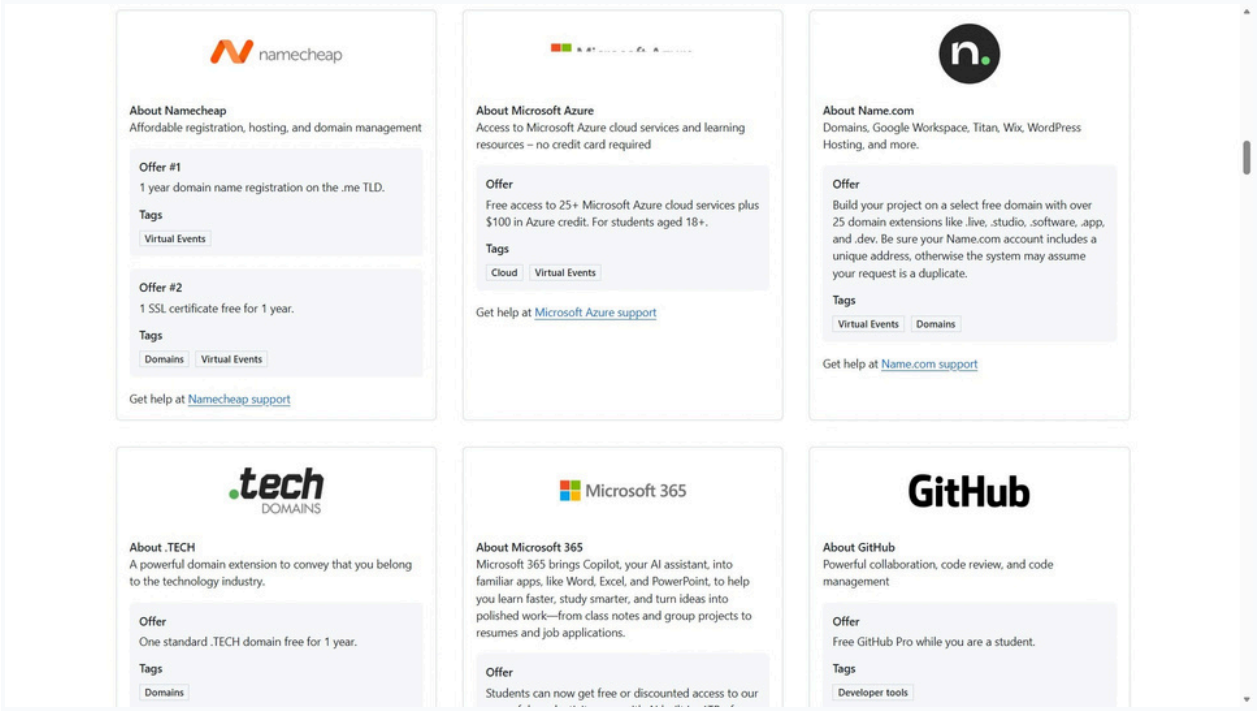
Part 3: The GitHub Student Developer Pack (Free Stuff!)

If you are a student, GitHub will give you thousands of dollars worth of professional software, tools, and website hosting completely **free**.

How to get it:

1. Go to education.github.com/pack.
2. Click **SignupforStudentDeveloper Pack**.

3. Log in with your GitHub account.
4. Upload proof that you are a student (like your college ID card or college email address).



What you get inside:

- A free website domain name (like .me for your personal portfolio).
- Free cloud server credits.
- Free professional coding apps (like JetBrains tools).

Part 4: Live Demo — How to Push Your First Code

Let's take code from your laptop and upload it to GitHub step-by-step:

1. Create a new folder on your desktop called my-first-project.
2. Inside that folder, create a file named index.html and type a simple line like <h1>Hello World</h1>.
3. Open **Git Bash** inside that folder and type these commands one by one:

```
git init
git status
git add index.html
git commit -m "My first commit!"
```

4. Go to GitHub, create a new empty repository, and copy its web link. Then link it in your terminal:
   `git remote add origin <PASTE-YOUR-GITHUB-LINK-HERE>`
   `git branch -M main`
   `git push-u originmain`
5. Refresh your GitHub page—congratulations, your code is now live on the internet!

```
devom@LAPTOP-F41A4IHJ MINGW64 ~/Desktop
$ cd beast-runner/

devom@LAPTOP-F41A4IHJ MINGW64 ~/Desktop/beast-runner (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

devom@LAPTOP-F41A4IHJ MINGW64 ~/Desktop/beast-runner (main)
$
```

Part 5: Practice Code Link

Want to practice right now? You can use this official practice repository to test out your cloning and pushing skills:

- **Practice Repo:** github.com/octocat/Spoon-Knife

Part 6: Where to Learn More?

If you want to have fun and master Git commands visually:

- **Play a Git Game:** Learn Git Branching (An interactive visual puzzle game for beginners).
- **Official Guides:** GitHub Skills (Step-by-step tutorials built right into GitHub).

